

Documentation Technique - Projet PAT Rufisque

1. Introduction

Ce document détaille l'architecture technique et le fonctionnement du site vitrine de la Politique Alimentaire Territoriale (PAT) de Rufisque.

Le site est une application web statique, conçue pour être performante, sécurisée et facile à maintenir. Il utilise des technologies web standard (HTML, CSS, JavaScript) et ne nécessite pas de base de données ou de langage serveur complexe pour son fonctionnement de base.

Technologies principales :

- **HTML5** pour la structure sémantique.
 - **CSS3** pour la mise en forme, avec le framework **Tailwind CSS** pour un design rapide et responsive.
 - **JavaScript (ES6+)** pour la logique applicative et la génération dynamique de contenu côté client.
 - **mviewer** pour la composante cartographique interactive.
-

2. Architecture Générale

L'architecture du site repose sur un principe de **génération de page côté client**. Le contenu principal de la page d'accueil est assemblé dynamiquement par le navigateur de l'utilisateur.

Ce modèle fonctionne comme suit :

1. **Le Squelette (index.html)** : La page `index.html` est un fichier HTML minimaliste qui sert de "coquille" ou de squelette. Il contient les conteneurs principaux (ex: `#main-content`, `#footer-container`) mais très peu de contenu visible.
2. **Les Données (contenu.json)** : Toute l'information textuelle, les titres, les descriptions, les liens vers les images et les vidéos sont centralisés dans le fichier `contenu.json`. Ce fichier agit comme une base de données légère pour le site.
3. **Les Gabarits (Partials)** : Le dossier `/partials` contient des fragments de code HTML (`.html`) qui servent de gabarits (templates) pour chaque section du site (accueil, axes stratégiques, actions, etc.).
4. **L'Assembleur (main-builder.js)** : Le script `js/main-builder.js` est le moteur de l'application. Au chargement de la page, il :
 - Récupère les données de `contenu.json`.
 - Charge les gabarits HTML depuis le dossier `/partials`.
 - Injecte les données dans les gabarits.
 - Assemble les sections complétées et les insère dans le squelette `index.html`.

Ce découplage rend la **mise à jour du contenu extrêmement simple**, car elle ne nécessite que la modification du fichier `contenu.json`, sans toucher au code HTML ou JavaScript.

3. Structure des Fichiers

```
.
├── css/                # Fichiers de style
│   ├── navbar.css
│   ├── shared-styles.css # Styles partagés
│   └── style.css
├── js/                 # Scripts JavaScript
│   ├── main-builder.js  # Cœur de l'application : assemble la page d'accueil
│   └── navbar.js         # Gère la logique de la barre de navigation
├── mviewer/            # Application cartographique mviewer intégrée
├── partials/           # Gabarits HTML pour les sections de la page
│   ├── accueil.html
│   ├── actions.html
│   └── ...
├── statics/            # Ressources statiques (images, PDF, etc.)
├── contenu.json         # Fichier central des données du site
├── index.html           # Point d'entrée principal (page d'accueil)
├── navbar.html          # Structure de la barre de navigation
└── page-*.html          # Pages de contenu spécifiques (axes stratégiques)
```

4. Gestion du Contenu

Pour modifier le contenu de la page d'accueil, il suffit d'éditer le fichier `contenu.json`.

Exemple : Modifier le titre de la section d'accueil

Dans `contenu.json`, localisez l'objet `accueil` :

```
"accueil": {
  "titre_prefix": "> Rufisque s'engage pour une politique alimentaire",
  "titre_highlight": "locale, saine et durable",
  "description": "Avec la <strong...>",
  ...
},
```

En modifiant les valeurs des clés `titre_prefix` ou `titre_highlight`, les changements seront automatiquement répercutés sur le site au prochain chargement.

5. Composants et Logique

`js/main-builder.js`

Ce script est le chef d'orchestre de la page d'accueil.

- `buildHomePage()` :
 - Utilise `Promise.all` pour charger en parallèle `contenu.json` et tous les gabarits HTML nécessaires.

- Définit une fonction `render()` qui remplace les marqueurs `{{cle}}` dans les gabarits par les données correspondantes du JSON.
- Construit le HTML final pour chaque section.
- Injecte le HTML assemblé dans les conteneurs de `index.html`.
- `initVideoFacade()` : Gère l'affichage de la vidéo YouTube. Une image de façade est affichée pour optimiser le temps de chargement. Au clic, l'image est remplacée par le lecteur vidéo embarqué.

js/navbar.js

Ce script gère tous les aspects de la navigation.

- Il charge le contenu de `navbar.html` dans le conteneur `#navbar-container` présent sur toutes les pages.
- Il détecte la page actuelle et applique une classe `.active-page` au lien correspondant pour le mettre en surbrillance.
- Il gère le comportement "collant" (sticky) de la barre de sous-navigation sur les pages des axes.
- Il implémente la logique d'ouverture/fermeture du menu mobile ("burger").

mviewer/

Ce répertoire contient une instance complète de l'application de cartographie web **mviewer**. Elle est configurée pour afficher des données géographiques spécifiques au projet PAT Rufisque (parcelles, cantines, etc.). Son intégration se fait via un `iframe` dans la page `axe_carte.html`.

6. Déploiement

Le projet est un ensemble de fichiers statiques. Pour le déployer, il suffit de copier l'ensemble des fichiers et dossiers (sauf les fichiers de configuration comme `.git`, `venv`, etc.) sur un serveur web.

- **Serveur compatible** : N'importe quel serveur HTTP (Apache, Nginx, etc.) peut héberger le site.
- **Services d'hébergement** : Le site est compatible avec les plateformes d'hébergement statique comme GitHub Pages, Netlify, Vercel, etc.

Aucune configuration côté serveur n'est requise.

7. Cartographie (mviewer)

La composante cartographique est gérée par **mviewer**. Les configurations spécifiques à ce projet se trouvent dans le dossier `mviewer/apps/public/`. Chaque sous-dossier (`cantines`, `gouvernance`, `zone_de_lendeng`) représente une carte thématique distincte, définie par un fichier de configuration XML principal.

Ces cartes utilisent des **couches personnalisées** (`customlayer`), qui sont des scripts JavaScript (`/mviewer/customlayers/*.js`) chargeant des données depuis des fichiers GeoJSON locaux.

7.1. Carte : Cantines Scolaires

- **Fichier de configuration** : `mviewer/apps/public/cantines/map_cantines.xml`
- **Titre** : "Un réseau de cuisines centrales"

- **Objectif** : Visualiser la localisation des cuisines centrales et des cantines scolaires qu'elles desservent.
- **Couches de données** :
 - **Cuisines centrales** (`cuisine_centrale.js`) : Affiche les points des cuisines. Les données proviennent de `cantines/cuisine_centrale.geojson`. Une infobulle et un panneau latéral affichent les détails au clic.
 - **Cantines scolaires** (`cantines_scolaires.js`) : Affiche les points des écoles. Les données proviennent de `cantines/cantines_scolaires.geojson`.

7.2. Carte : Gouvernance (Producteurs locaux)

- **Fichier de configuration** : `mviewer/apps/public/gouvernance/map_gouvernance.xml`
- **Titre** : "Un réseau de producteurs locaux"
- **Objectif** : Cartographier les fournisseurs et producteurs locaux par secteur d'activité.
- **Couches de données** :
 - Les couches sont regroupées par **thème**. Le thème principal "Fournisseurs locaux par activité" est activé par défaut.
 - Chaque couche représente un type d'activité : `Transformation de céréales`, `Maraichage / agriculture`, `Aviculture`, `Boucherie`, `Pisciculture`, `Boutiques`.
 - Chacune de ces couches est un `customlayer` (`fournisseurs_*.js`) qui filtre les données du fichier principal `gouvernance/fournisseurs.geojson` pour n'afficher que les points correspondants à son activité.

7.3. Carte : Zone Agricole de Lendeng

- **Fichier de configuration** : `mviewer/apps/public/zone_de_lendeng/zone_de_lendeng.xml`
- **Titre** : "Zone agricole de Lendeng"
- **Objectif** : Fournir une vue détaillée des aménagements et des ressources de la zone agricole de Lendeng.
- **Couches de données** :
 - Cette carte est la plus complexe, avec de nombreuses couches KML et GeoJSON chargées via des scripts `customlayer` situés directement dans le dossier de l'application (`zone_de_lendeng/*.js`).
 - **Thèmes** :
 - **Ressources en eau** : `Voies de ruissellement`, `Zones inondables`, `Bassin de rétention`.
 - **Aménagements hydrauliques** : `Pompes`, `Points d'alimentation`, `Réseau d'alimentation`.
 - **Limites de la zone** : `Voies d'accès`, `Limite de la zone cultivée`, `Surfaces cultivées`.
 - Les données proviennent de multiples fichiers `.kml` et `.js` dans le même répertoire.